



Univerza v Mariboru



Fakulteta za elektrotehniko,
računalništvo in informatiko

ATURP Seminarska naloga:

Bančna kartica

Jernej Lobnik, Jakob Renčelj, Žan Misja

Maribor, maj 2026

KAZALO

1	UVOD	1
2	OPIS REŠITVE Z METODO GROBE SILE	3
2.1	Postopek reševanja	3
2.2	Psevdokoda	4
2.3	Robni primeri	4
3	OPIS REŠITVE S PRIMERNO STRATEGIJO	5
3.1	Postopek reševanja	5
3.2	Psevdokoda	5
3.3	Robni primeri	6
4	ZGLED TEŽJE INSTANCE PROBLEMA	7
4.1	Vizualizacija instance	7
4.2	Metoda grobe sile	7
4.3	Primerna strategija	7
5	PREDSTAVITEV IN PRIMERJAVA REZULTATOV	9
5.1	Ugotovitve:	9
6	ZAKLJUČEK	6
7	NAVAJANJE VIROV	7

KAZALO SLIK

SLIKA 1 PRIMER ENEGA KORAKA	5
SLIKA 2 PSEVDOKODA OPTIMALNE REŠITVE	6
SLIKA 3 PRIMERJAVA ALGORITMOV GLEDE NA ČASOVNO ZAHTEVNOST	9

KAZALO TABEL

TABELA 1 KONCEPTUALNI PRIKAZ TABELE	7
---	---

1 UVOD

Sprehajate se po lepem parku. V parku najdete bančno kartico, za dostop do denarja na kartici potrebujete PIN kartice. PIN koda ima 4 števke med 0 in 9, torej je samo 10000 možnih kombinacij. To niti ni tako veliko. Odpravite se do bankomata in poskusite kar z 1111. Začuda je PIN koda delovala. Seveda so današnji varnostni standardi zelo drugačni.

V nalogi imamo problem generiranja varnih PIN kod. PIN mora vsebovati natanko **P** števk, pri čemer je vsaka številka iz **N**-tiškega številskega sistema. Poleg tega mora PIN vsebovati vsaj **K** različnih števk, saj želimo preprečiti preveč enostavne kombinacije, kot so na primer 1111, 0000 ali 2222.

Podoben problem se pojavlja tudi v resničnem svetu pri načrtovanju gesel in varnostnih sistemov. Številne spletne storitve zahtevajo, da geslo vsebuje različne tipe znakov, kot so velike črke, male črke, številke in posebni znaki. Namen takšnih pravil je povečanje števila možnih kombinacij in s tem izboljšanje varnosti sistema.

Vhodni podatki naloge so tri cela števila:

- **N** – število različnih števk v številskem sistemu,
- **K** – minimalno število različnih števk v PIN kodi,
- **P** – dolžina PIN kode.

Če problem opišemo matematično, iščemo število nizov dolžine **P**, kjer je vsak element iz množice: $\{0, 1, 2, \dots, N-1\}$, niz mora vsebovati vsaj **K** različnih elementov, drugače PIN-a ni mogoče sestaviti.

Primer enostavne instance problema: $N = 3$, $K = 2$, $P = 2$

Možne številke so: $\{0, 1, 2\}$

Vsi možni PIN-i dolžine 2 so:

00, 01, 02,

10, 11, 12,

20, 21, 22

Ker mora PIN vsebovati vsaj 2 različni števki, ostanejo veljavni:

01, 02,

10, 12,

20, 21

Torej je rezultat za ta primer enak 6.

2 OPIS REŠITVE Z METODO GROBE SILE

Pri rešitvi z uporabo grobe sile preverimo vse možne PIN kombinacije in za vsako posebej ugotovimo ali ustreza pogojem naloge. Torej ali posamezna PIN koda vsebuje več kot K različnih števk.

Število vseh možnih PIN kombinacije je: N^P , saj ima vsaka izmed P pozicij natanko N možnih vrednosti.

2.1 Postopek reševanja

Naša rešitev deluje po teh korakih:

1. Izračunamo število vseh možnih PIN kombinacij
2. Za vsako število ustvarimo ustrezen PIN
3. Število pretvorimo v zapis v N-tiški sistem
4. Preštejemo različne števke v PIN-u
5. Če PIN vsebuje vsaj K različnih števk se števec poveča
6. Rezultat delimo po modulu in izpišemo

Vsako število pretvorimo v PIN tako, da večkrat uporabimo operaciji:

ostanek = število mod N

število = število / N

Primer: $N = 5$, $P = 3$

število = 17

Pretvorba:

$17 \bmod 5 = 2$

$17 / 5 = 3$

$3 \bmod 5 = 3$

$3 / 5 = 0$

$0 \bmod 5 = 0$

Dobimo PIN:

230

kar predstavlja eno izmed možnih kombinacij v petiškem sistemu.

2.2 Psevdokoda

funkcija izracunajSteviloPINov(N, K, P):

odgovor = 0

steviloMoznosti = N^P

za i od 0 do steviloMoznosti:

temp = i

pin = prazen seznam dolžine P

za j od 0 do P:

pin[j] = temp mod N

temp = temp / N

razlicne = množica vseh števk iz pin

če je velikost(razlicne) $\geq K$:

odgovor = odgovor + 1

vrni odgovor mod ($10^9 + 7$)

2.3 Robni primeri

- Če je $K = 1$ so veljane vse PIN kode.
- Če je $K > P$, rešitev ne obstaja
- Če je $N = 1$ obstaja samo ena možna rešitev

Pomanjkljivosti:

Pri večjih številih N in P , program ne deluje, saj ne more kalkulirati N^P . Na primer, če je $N = 999$ in $P = 1337$, bi program moral izračunati 999^{1337} , kar je ogromna številka. Številka je tako velika, da je niti ne moremo shraniti. Zato program pri večjih vhodnih podatkih ne deluje.

Časovna zahtevnost:

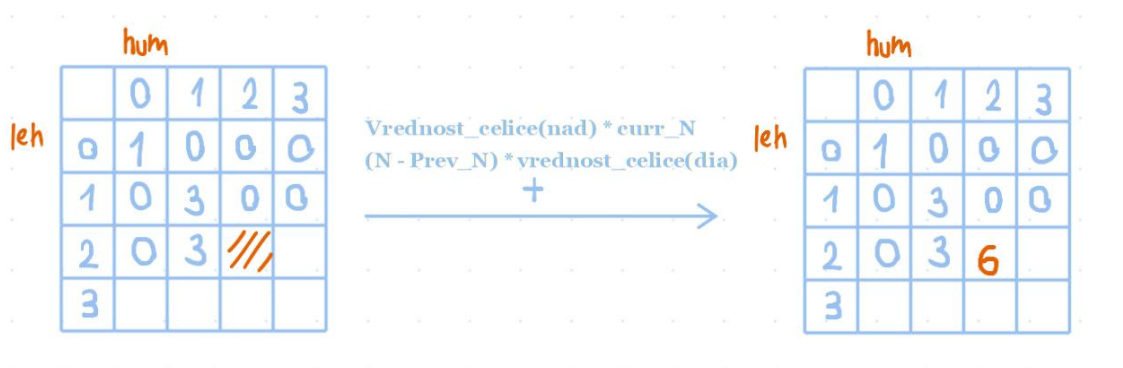
Metoda pregleda vse PIN-e (N^P) in za vsak PIN kodo preveri še število različnih števk zato je skupna časovna zahtevnost približno: $O((N^P) * P)$.

3 OPIS REŠITVE S PRIMERNO STRATEGIJO

Primerne strategije smo se lotili z uporabo dinamičnega programiranja[1]. Strategija deluje tako da sproti gradi gesla velikosti L in se na vsake koraku odloči ali vzame novo cifro, ali eno ki smo jo že izbrali prej.

3.1 Postopek reševanja

1. Inicializacija 2D tabele DP
2. Iteracija skozi tabelo:
3. Izračun možnosti z že uporabljenimi števki (celica nad)
4. Izračun možnosti z novimi števki (celica diagonalno)
5. Določitev vrednosti trenutne celice
6. Izračun končnega rezultata



Slika 1 Primer enega koraka
[2]

3.2 Psevdokoda

Najprej pred samim začetkom algoritma ustvarimo 2D tabelo ustrezne velikosti. Skozi to tabelo se sistematično pomikamo po vrsticah od zgoraj navzdol in po stolpcih od leve proti desni.

Pri izračunu vsake celice združimo dve možnosti:

1. Vzamemo vrednost celice nad trenutno in jo pomnožimo s trenutno številko stolpca. To predstavlja primere, kjer za naslednje mesto v PIN-u ponovno uporabimo številko, ki smo jo že izbrali v preteklosti.

2. Nato vzamemo še vrednost celice diagonalno levo zgoraj in jo pomnožimo s številom preostalih izbir. S tem dobimo vse možnosti za primere, ko PIN-u dodamo povsem novo, še neuporabljeno številko.

Ti dve vrednosti seštejemo in rezultat zapišemo v trenutno celico. Ko je tabela napolnjena, pogledamo njeno zadnjo vrstico. Končni rezultat dobimo tako, da v zadnji vrstici seštejemo vrednosti vseh celic od stolpca K naprej proti desni, saj stolpci predstavljajo število različnih cifr v trenutni dolžini PIN-a

```

3  funkcija OptimalAlgoritem(DP)
4      output = 0
5
6      za vrstico od 1 do P:
7          za stolpec od 1 do N:
8
9              curr_n = stolpec
10             prev_n = stolpec - 1
11
12             vrednost_celice(trenutno) = vrednost_celice(nad) * curr_n
13             vrednost_celice(trenutno) += vrednost_celice(dia) * (N - prev_n)
14
15             vrednost_celice(trenutno) = vrednost_celice(trenutno) mod (10^9 + 7)
16
17         za stolpec od K do N:
18             output += vrednost_celice(zadnja_vrstica, trenutni_stolpec)
19             output = output mod (10^9 + 7)
20
21     vrni output

```

Slika 2 Psevdokoda optimalne rešitve

3.3 Robni primeri

Pri razvoju algoritma smo morali upoštevati naslednje robne primere:

- $K > N$: Če je zahtevano število različnih števk večja od vseh možnih števk, je rezultat vedno 0.
- $K > P$: Če je zahtevano številko različnih števk večja od dolžine PIN-a, je rezultat vedno 0.
- Zaradi eksponentne rasti kombinacij se vsak delni izračun izvaja po modulu 10^9+7

4 ZGLED TEŽJE INSTANCE PROBLEMA

Za testiranje mejnih zmogljivosti algoritma smo izbrali parametre, ki predstavljajo zgornjo mejo zahtevnosti naše naloge:

- **Število možnih števk (N):** 2024
- **Zahtevano število različnih števk (K):** 1500
- **Dolžina PIN-a (P):** 2024
- **Modul:** 10^{9+7}

4.1 Vizualizacija instance

Pri takšnih parametrih ne moremo vizualizirati celotne 2D tabele, saj bi imela celotna tabela preko 4 milijone celic (2024×2024). Spodaj je konceptualni del tabele:

PIN	1	2	3	...	2024
1	2024	0	0	...	0
2	2024	4094552	0	...	0
...	...	...	...	...	...
2024	(Izračun)	(Izračun)	(Izračun)	...	(Izračun)

Tabela 1 Konceptualni prikaz tabele

4.2 Metoda grobe sile

Pri metodi grobe sile pride do treh težav:

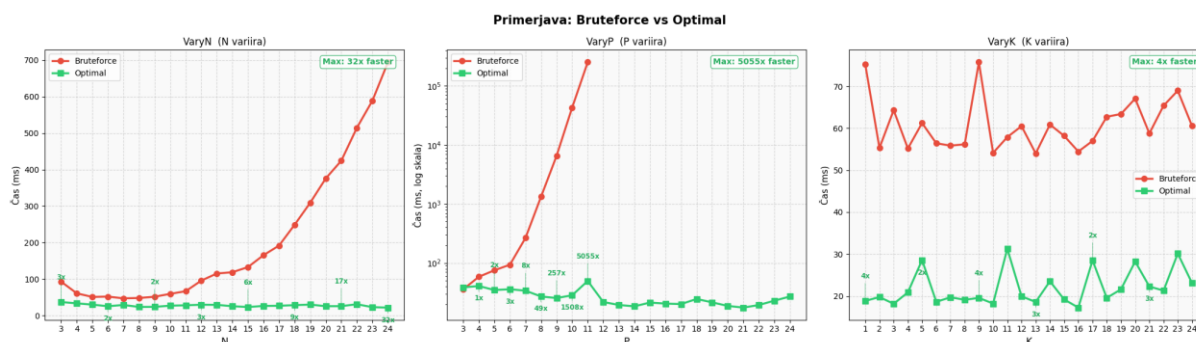
- **Število kombinacij:** Algoritem mora izračunati N^P kombinacij, kar je v našem primeru 2024^{2024} kar je številka, ki ima skoraj 7000 cifer.
- **Časovna zahtevnost:** Algoritem ima eksponentno zahtevnost ($O(N^P)$), kar pa postane hitro neobvladljivo.
- **Omejitve podatkovnih tipov:** Največji standardni tip v C++ lahko hrani približno 1.8×10^{19} veliko število, kar je zanemarljivo v primerjavi z 2024^{2024} .

4.3 Primerna strategija

Primerna strategija deluje v polinomskem času ($O(P * N)$), saj imamo toliko izračunov, kolikor je elementov v tabeli, kar je pri naši instanci samo približno 4 milijone.

- **Število operacij:** Izvede približno 4 milijone enostavnih aritmetičnih operacij.
- **Čas izvajanja:** Sodoben računalnik lahko to izračuna v nekaj manj kot 0.1 sekunde.
- **Obvladovanje velikih števil:** Ker algoritem uporablja modul pri vsakem koraku, se izognemo problemu prevelikih števil.

5 PREDSTAVITEV IN PRIMERJAVA REZULTATOV



Slika 3 Primerjava algoritmov glede na časovno zahtevnost

Za primerjavo algoritmov smo izmerili čas izvajanja v odvisnosti od vhodnih parametrov N, K in P.

Testni primeri: Testirali smo tri scenarije:

- variiranje parametra N (K=2, P=4 fiksna), vrednosti N od 2 do 24
- variiranje parametra P (N=5, K=2 fiksna), vrednosti P od 2 do 24
- variiranje parametra K (N=6, P=2 fiksna), vrednosti K od 1 do 24

Za vsako kombinacijo smo izmerili čas izvajanja obeh algoritmov. **Meritev bruteforce algoritma smo preskočili, ko je čas presegel 10 minut.** To je tudi razvidno iz sredinskega grafa, kjer bruteforce ni obdelal zahtevnejših istanc problema.

5.1 Ugotovitve:

Iz grafov je razvidno, da parameter P najbolj vpliva na čas izvajanja bruteforce algoritma saj, pri P=9 čas preseže 1000 ms in eksponentno narašča do vrednosti nad 10⁵ ms pri P=11+, kjer so bile meritve preskočene. Maksimalna izmerjena pohitritev optimalnega algoritma glede na bruteforce in dane razmere merjenja je bila pri variiranju P **5055x**. Parameter N ima polinomičen vpliv na bruteforce, kjer smo dosegli pohitritev **32x** pri N=24. Parameter K praktično ne vpliva na čas izvajanja bruteforce algoritma, saj časi ostajajo med 55–70 ms ne glede na vrednost K, pohitritev pa je le **4x**.

Optimalni algoritem ostaja stabilen pri vseh testiranjih, kar pomeni da so časi podobni ne glede na vrednosti parametrov in s tem potrjuje polinomično časovno zahtevnost $O(N \cdot P)$.

Ocena časovne zahtevnosti:

- Bruteforce: $O(N^P \cdot P)$ → eksponentna rast, neuporabno pri večjih vrednostih P
- Optimal: $O(N \cdot P)$ → polinomična, praktično linearna

6 ZAKLJUČEK

V nalogi smo obravnavali problem generiranja varnih PIN kod, kjer PIN dolžine P vsebuje števke iz N -tiškega številskega sistema in mora vsebovati vsaj K različnih števk. Cilj je bil izračunati število vseh veljavnih kombinacij po modulu 10^9+7 .

Implementirali smo dve rešitvi. Metoda grobe sile pregleda vse N^P možnih kombinacij in za vsako preveri pogoj. Njena časovna zahtevnost je $O(N^P \cdot P)$, kar jo naredi neuporabno pri večjih vhodnih podatkih, saj že pri $N=5$, $P=11$ čas preseže 1000 ms in eksponentno narašča. Optimalna rešitev temelji na dinamičnem programiranju, kjer gradimo 2D tabelo, ki beleži število PIN-ov dolžine p z natanko n različnimi števki. Njena časovna zahtevnost je $O(N \cdot P)$, kar je polinomično in praktično linearno.

Meritve so potrdile teoretično analizo. Pri variiranju parametra P je optimalni algoritem dosegel do 5055x pobitritev glede na brute force, pri variiranju N pa do 32x. Parameter K nima bistvenega vpliva na čas izvajanja nobene od rešitev, saj ne zmanjša števila iteracij.

Optimalna rešitev uspešno reši tudi težjo instanco problema ($N=2024$, $K=1500$, $P=2024$), ki je za metodo grobe sile popolnoma nedosegljiva, saj bi morala izračunati 2024^{2024} kombinacij, medtem ko dinamično programiranje izvede le ~ 4 milijone aritmetičnih operacij v manj kot 0.1 sekunde.

7 NAVAJANJE VIROV

- [1] »Dynamic Programming or DP«, GeeksforGeeks. Pridobljeno: 19. maj 2026. [Na spletu]. Dostopno na: <https://www.geeksforgeeks.org/dsa/dynamic-programming/>
- [2] »Steps to solve a Dynamic Programming Problem«, GeeksforGeeks. Pridobljeno: 19. maj 2026. [Na spletu]. Dostopno na: <https://www.geeksforgeeks.org/dsa/solve-dynamic-programming-problem/>